

ЛР14 (3 пары / 6 часов)

ЛР14 (3 пары / 6 часов)

«Юнит-тесты: проверка TaskValidator и TaskService (Add/Update/Delete)»

0) Итог ЛР14 (что должно быть готово)

- ✓ В решении создан проект `TaskTracker.Tests`
 - ✓ Тесты запускаются и проходят
 - ✓ Есть минимум:
 - 3 теста для `TaskValidator`
 - 3 теста для `TaskService`
 - ✓ Добавлен отчёт `docs/LR14_Report.md`
 - ✓ Минимум 3 коммита за лабораторную
-

1) Теория (важное)

1.1. Unit test

Юнит-тест — проверка одной функции или одного правила.

Тест должен быть коротким, повторяемым и независимым.

1.2. Структура AAA

- Arrange — подготовка данных
 - Act — действие
 - Assert — проверка результата
-

2) Практика. Пара 1 (2 часа) — проект тестов и тесты валидатора

Шаг 1. Создать тестовый проект

В Visual Studio:

1. Solution → Add → New Project
 2. Выбрать **MSTest Test Project**
 3. Название: `TaskTracker.Tests`
 4. Create
-

Шаг 2. Добавить ссылку на Core

В проекте `TaskTracker.Tests`:

1. Dependencies → Add Project Reference
 2. отметить `TaskTracker.Core`
 3. OK
-

Шаг 3. Создать папку `ValidatorTests`

В `TaskTracker.Tests` создать папку:

- `ValidatorTests`
-

Шаг 4. Создать файл `TaskValidatorTests.cs`

Создать файл и вставить:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using TaskTracker.Core.Models;
using TaskTracker.Core.Validation;

namespace TaskTracker.Tests.ValidatorTests;

[TestClass]
```

```

public class TaskValidatorTests
{
    [TestMethod]
    public void Validate_TitleEmpty_ReturnsError()
    {
        var task = new TaskItem { Title = "", Description =
"" };
        var error = TaskValidator.Validate(task);
        Assert.IsNotNull(error);
    }

    [TestMethod]
    public void Validate_TitleTooLong_ReturnsError()
    {
        var longTitle = new string('a', TaskValidator.TitleMa
xLength + 1);
        var task = new TaskItem { Title = longTitle, Descript
ion = "" };
        var error = TaskValidator.Validate(task);
        Assert.IsNotNull(error);
    }

    [TestMethod]
    public void Validate_ValidTask_ReturnsNull()
    {
        var task = new TaskItem
        {
            Title = "Купить тетрадь",
            Description = "Описание",
            Status = TaskStatus.New
        };

        var error = TaskValidator.Validate(task);
        Assert.IsNull(error);
    }
}

```

```
}  
}
```

Шаг 5. Запустить тесты

Открыть Test Explorer и нажать **Run All**.

Шаг 6. Коммит №1

Сообщение коммита:

- `test: add MSTest project and TaskValidator tests`

Commit → Push.

3) Практика. Пара 2 (2 часа) — тесты TaskService

Шаг 7. Создать папку `ServiceTests`

Создать папку:

- `ServiceTests`

Шаг 8. Создать файл `TaskServiceTests.cs`

Создать файл и вставить:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;  
using TaskTracker.Core.Services;  
  
namespace TaskTracker.Tests.ServiceTests;  
  
[TestClass]  
public class TaskServiceTests  
{  
    [TestMethod]  
    public void Add_ValidTitle_TaskAddedWithId1()  
    {  
    }  
}
```

```

        var service = new TaskService();
        var task = service.Add("Test task");

        Assert.AreEqual(1, task.Id);
        Assert.AreEqual("Test task", task.Title);
        Assert.AreEqual(1, service.GetAll().Count);
    }

    [TestMethod]
    public void Add_EmptyTitle_Throws()
    {
        var service = new TaskService();
        Assert.ThrowsException<ArgumentException>(() => service.Add(""));
    }

    [TestMethod]
    public void Delete_ExistingId_RemovesTask()
    {
        var service = new TaskService();
        var task = service.Add("To delete");

        service.Delete(task.Id);

        Assert.AreEqual(0, service.GetAll().Count);
    }

    [TestMethod]
    public void Update_ChangesTitleAndDescription()
    {
        var service = new TaskService();
        var task = service.Add("Old");

        var updated = service.Update(task.Id, "New", "Desc");

        Assert.AreEqual("New", updated.Title);
    }

```

```
        Assert.AreEqual("Desc", updated.Description);
    }
}
```

Шаг 9. Запустить тесты

Test Explorer → Run All.

Шаг 10. Коммит №2

Сообщение коммита:

- `test: add TaskService unit tests`

Commit → Push.

4) Практика. Пара 3 (2 часа) — расширение тестов и отчёт

Шаг 11. Добавить тест на длинное Description

В `TaskValidatorTests.cs` добавить:

```
[TestMethod]
public void Validate_DescriptionTooLong_ReturnsError()
{
    var longDesc = new string('b', TaskValidator.DescriptionM
axLength + 1);
    var task = new TaskItem { Title = "Ok", Description = lon
gDesc };

    var error = TaskValidator.Validate(task);
    Assert.IsNotNull(error);
}
```

Шаг 12. Создать отчёт `docs/LR14_Report.md`

Создать файл и вставить:

```
# Отчёт ЛР14 — юнит-тесты

## Что сделано
- Создан проект TaskTracker.Tests (MSTest)
- Добавлены тесты TaskValidator:
  - пустой Title -> ошибка
  - слишком длинный Title -> ошибка
  - слишком длинное Description -> ошибка
  - валидная задача -> ok
- Добавлены тесты TaskService:
  - Add валидной задачи
  - Add пустого Title -> исключение
  - Delete существующей задачи
  - Update меняет Title и Description

## Как запускать тесты
Visual Studio -> Test -> Test Explorer -> Run All

## Результат
Все тесты проходят успешно.
```

Шаг 13. Коммит №3

Сообщение коммита:

- `test: extend tests and add LR14 report`

Commit → Push.

5) Что сдавать по ЛР14

- ✓ ссылка на репозиторий
- ✓ проект `TaskTracker.Tests`
- ✓ `docs/LR14_Report.md`

✓ скрин Test Explorer (зелёные тесты)
